

Predicting Molecular Hydrogen Density (nH₂) in RAMSES Simulations Using Machine Learning

Project Goal

To test whether machine learning can predict molecular hydrogen density in RAMSES simulations accurately enough to be useful as a surrogate model, including under changes in UV field strength and when generalizing from only part of a given volume.

We tested several machine learning approaches on seven 128x128x128 RAMSES simulation cubes spanning $G_0=0.1$ to 6.4, using local physical features derived from the simulations. The main result is that this is possible: the best models achieved strong performance in the main leave-one-cube-out test and remained useful in additional extrapolation and partial-volume reconstruction experiments. Among the methods tested, simpler pointwise models augmented with engineered spatial features performed best.

Dataset

Seven RAMSES simulations, each a 128x128x128 cube for a total of ~15M cells. The simulations differ only by the UV field strength G_0 with the values 0.1, 0.2, 0.4, 0.8, 1.6, 3.2, 6.4. Raw data contained 18 fields, reduced to 15 features: log-transformed densities, temperature, and self-shielding, extinction, G_0 , three-component velocity, six-faced magnetic field.

Models and Feature Engineering

We evaluated multiple models:

Family	Variants	Purpose
Linear Regression	-	Baseline
XGBoost (gradient-boosted decision trees)	Depth 4/6/8	Tests feature interaction order, considered excellent in tabular data like this dataset.
Fully connected MLP	Standard / wide / residual	Tests whether network size or skip connections help
3D U-Net	Base-ch 16/32/64	Tests whether learned spatial filters are enough

XGBoost-guided CNN	-	Adds XGBoost prediction as 16th feature
Equal-weight ensemble	XGBoost + MLP	Averages two effective methods
Ridge-stacked ensemble	-	Test whether learning a weighted average beats a simple average

Spatial Neighborhood Features

Pointwise models (XGBoost, MLP) see each cell in isolation but nH2 has inherent spatial dependency. To avoid having to rely on a CNN learning the spatial information which is computationally heavy, we precomputed 3D box-filter averages of all 15 features at three scales (3^3 , 5^3 , 7^3) giving us another 45 spatial features for a total of a 60-dimensional feature vector per cell.

Augmentation

The dataset is problematic for CNN training as each entire simulated cube is the effective input, meaning we had only 6 training samples (7th reserved for testing). 8 rotation operations were introduced creating 48 total training samples.

Limitations

Due to hardware constraints (the project was run on a personal computer, which was equipped with an AMD 5950X cpu, 32 gb DDR4 ram, and an Nvidia RTX 3090 (24 GB VRAM)) the CNN runs were run downsampled to 64x64. A future continuation could be to attempt this with a dedicated AI processing unit such as the Nvidia A100, which I did not have access to.

Initial Experiment and Results

The initial test was performed by training each of the variants below on 6 of the 7 simulations and attempting a prediction on the 7th. This tests the models' abilities to learn the underlying physics as they are affected by the change in UV intensity.

Variant	Mean R ²	Std	G0=0.1	G0=0.2	G0=0.4	G0=0.8	G0=1.6	G0=3.2	G0=6.4
XGBoost + MLP, 0.5/0.5	0.9482	0.024	0.908	0.915	0.953	0.962	0.967	0.967	0.965
Ridge	0.9456	0.020	0.912	0.916	0.954	0.960	0.962	0.961	0.954

Stacked									
XGB standard	0.9243	0.026	0.912	0.874	0.920	0.934	0.924	0.948	0.959
MLP wide	0.9127	0.034	0.843	0.886	0.914	0.928	0.946	0.945	0.927
XGB standard (no spatial)	0.8864	0.061	0.891	0.751	0.866	0.902	0.910	0.932	0.952
MLP standard (no spatial)	0.8862	0.046	0.809	0.825	0.883	0.921	0.929	0.928	0.910
Best 3D U-Net	0.803	0.172	0.497	-	-	-	-	-	0.591
Linear Regression	0.230	-	-0.876	-0.237	-	-	-	-	-

Final Model Improvement Progression

Configuration	Features	Mean R^2	Change
XGB alone	15	0.886	-
+ Spatial 3^3	30	0.917	+0.031
+ 5^3 + 7^3	60	0.924	+0.007
+ MLP ensemble	60	0.948	+0.024

Neural architecture (MLP width, residual blocks) and tree depth produced negligible improvements.

The 3D UNet was consistently worse than pointwise models and significantly more variable. This is likely due to only having 48 volumetric samples.

Supplementary Experiments and Results

Using the winning model from the initial experiment, the stacked XGBoost + MLP ensemble, two further experiments were carried out

Single-cube Extrapolation

Training the stack on a single G0 cube and predicting all 7 cubes, repeated for each of the cubes. The full results are available in the presentation. On average:

Steps from training cube	Typical R ² (approximately)
1 step (e.g 0.1 -> 0.2)	0.97
2	0.96
3	0.88
4	0.75
5	0.45
6	0.15

Extrapolation is significantly harder when learning at high UV strengths and attempting predictions at lower strengths.

Intra-cube Spatial Section

Instead of training on an entire simulation box, we trained the model on only part of the volume, and asked it to reconstruct the original full volume. Three filter geometries were tested: random voxels, halves of the volume, and contiguous boxes at varying fractions of the total volume.

Training Filter Geometry	Volume Fraction	R ²
Random Voxels	1%	-1.737 to 0.897
Random Voxels	5%	0.649 to 0.952
Random Voxels	10%	0.963 to 0.986
Contiguous Box	10%	0.757 to 0.955
Half-volume	50%	-5.050 to 0.935

The main finding is that a 1% random selection of voxels outperforms half-volume in most cases and is significantly less variable. As little as 5% either in randomly selected voxels or a contiguous box is enough to generalize to the total volume quite well.

Visuals of the predictions with comparisons to the simulation are available in the presentation.

Takeaways

- Machine learning methods can successfully predict nH₂ from other local simulation features
- The results suggest that ML can provide useful approximate predictions when full simulations are considered too costly
- Generalization across UV strengths is good for nearby conditions and degrades as extrapolation becomes more extreme
- The strongest improvement came from adding spatial neighborhood features, showing local spatial context is important
- Pointwise models + engineered spatial features outperformed 3D CNNs
 - Within the limitations, CNNs are able to capture the geometric structure but fail at resolving detailed values
- Feature engineering beats architecture exploration
 - The single largest gain came from adding the spatial features

Possible Future Work

- Explore whether this solution can be generalized
 - Test on SILCC simulations
- Explore additional ML methods
 - PINNs (Physics-Informed Neural Networks)
 - FNOs (Fourier Neural Operators)
 - More complex ensembles
- Test temporal predictions instead of spatial
 - Could ML be used to evolve a given simulation snapshot in time?
- Extend observational data
 - Treat observations as training data and use ML to predict outside the observations